Computación Paralela y Distribuída

2026-II

José Fiestas PhD

16 de septiembre de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 4: Comunicación y coordinación

Objetivos:

1. Pasos de Mensaje: MPI, Mensajes Punto a Punto, MPI, Comunicación Colectiva,
2. Blocking vs non-blocking
3. Memoria Compartida: OMP, Constructores y cláusulas
4. Programacion Hibrida

Temática

OpenMP (Open Multi-processing)

OpenMP (Open Multi-processing)

OpenMP es un API (Application Programming Interface), que provee un modelo escalable, portable para aplicaciones de memoria compartida. Es aplicable en C/C++ y Fortran, en diferentes arquitecturas. **OpenMP** utiliza directivas de compilador, librerias y variables de entorno

Fecha	Versión
Oct 1997	Fortran 1.0
Oct 1998	C/C++ 1.0
Nov 1999	Fortran 1.1
Nov 2000	Fortran 2.0
Mar 2002	C/C++ 2.0
May 2005	OpenMP 2.5
May 2008	OpenMP 3.0
Jul 2011	OpenMP 3.1
Jul 2013	OpenMP 4.0
Nov 2018	OpenMP 5.0
Nov 2020	OpenMP 5.1
Nov 2021	OpenMP 5.2
Nov 2024	OpenMP 6.0



Uso en 2008:
11 compañías y 5 centros HPC

Uso en 2020:
11 compañías y 22 centros HPC

OpenMP (Open Multi-processing)

OpenMP es portable en maquinas multi-procesos y de memoria compartida (UMA, o ccNUMA).

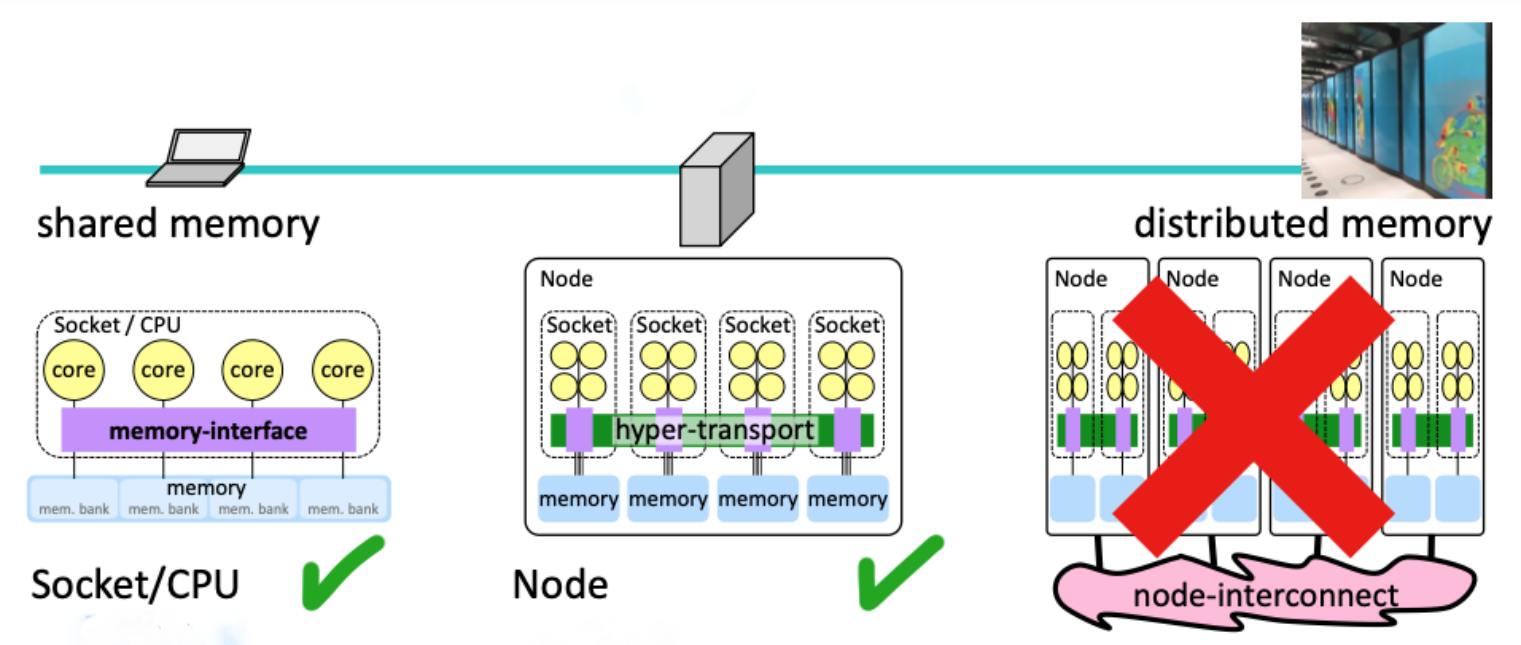
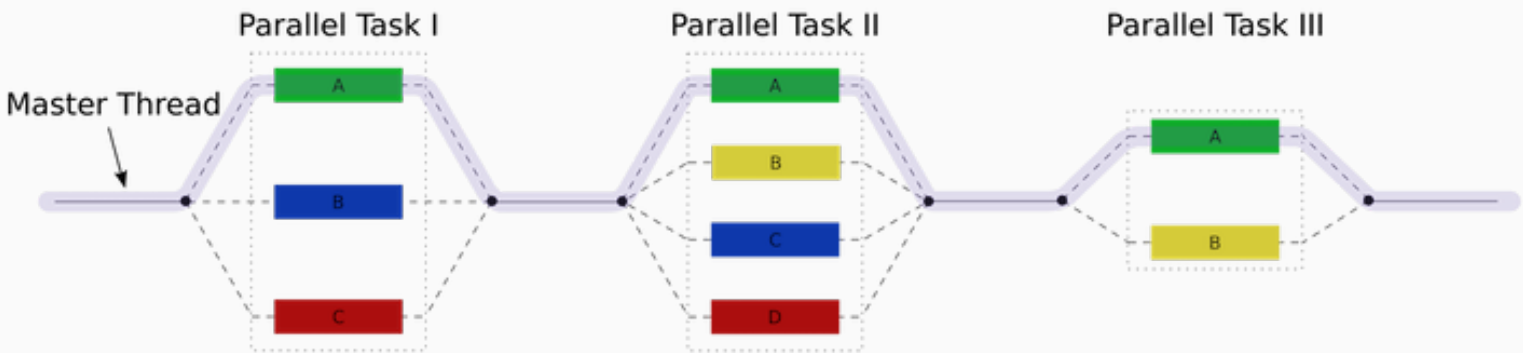


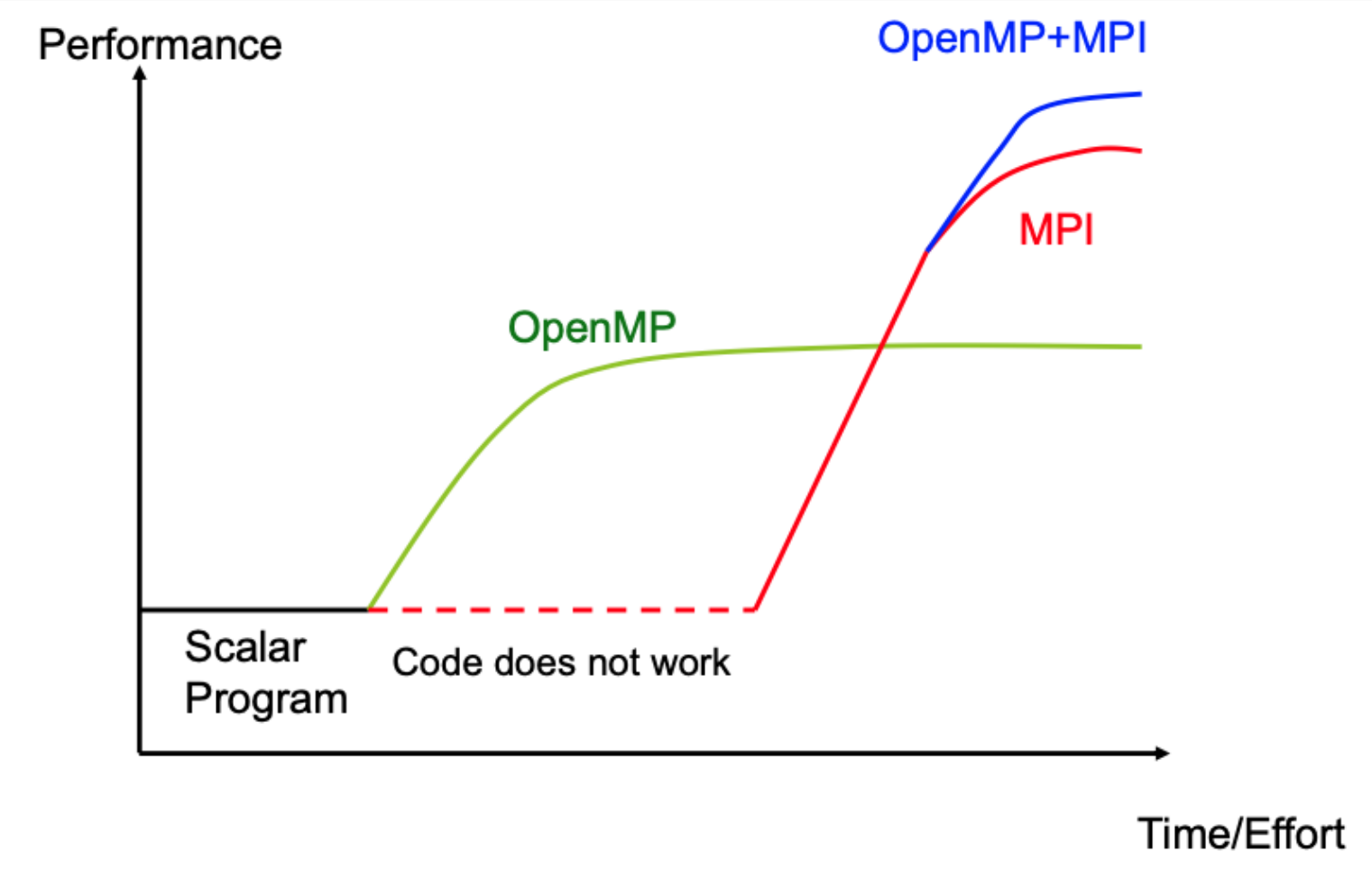
Figura 1: Arquitecturas de aplicación con OMP

Modelo Fork-Join:

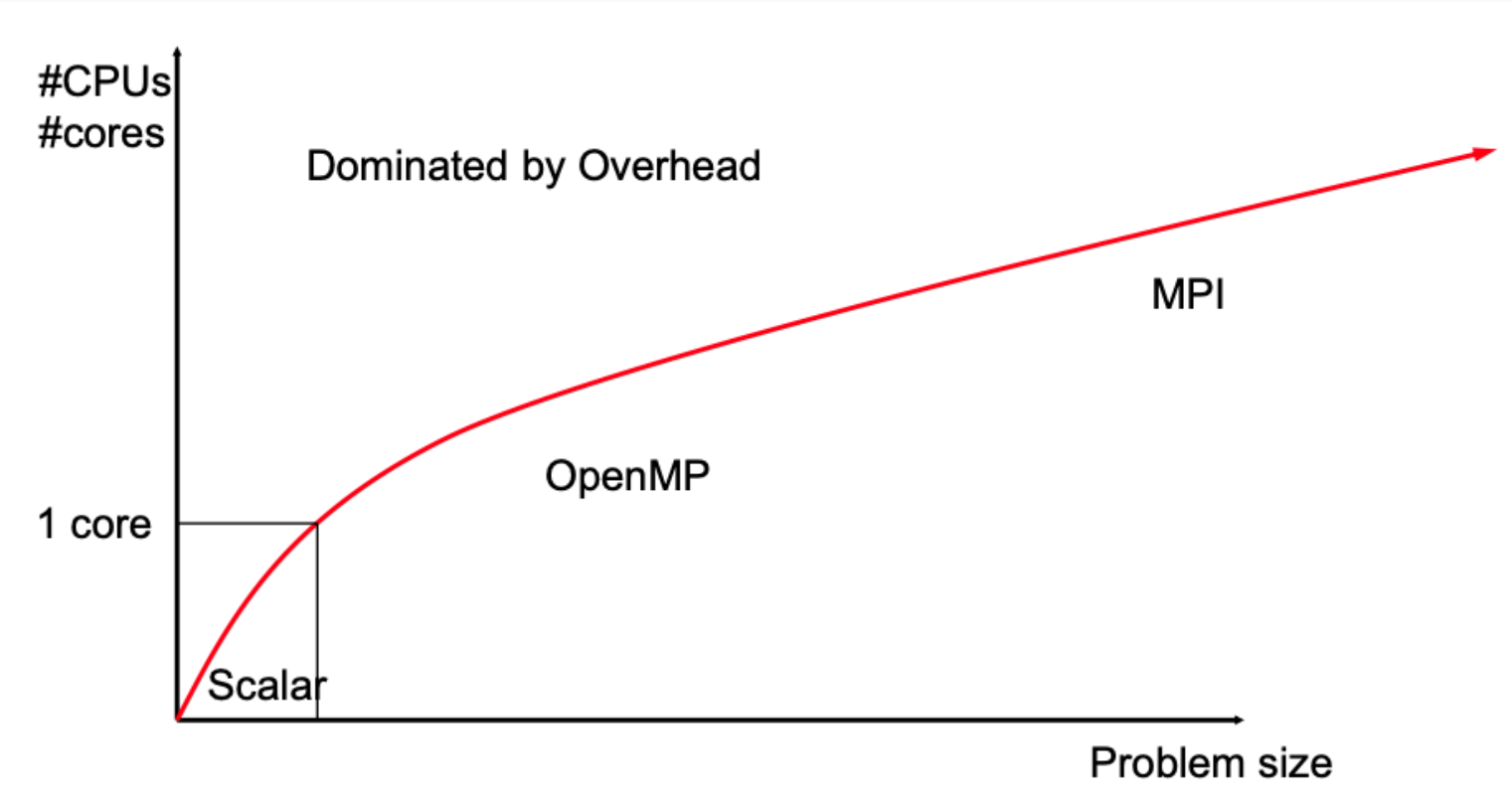
OpenMP paraleliza usando hilos de ejecución (*threads*): unidad de procesamiento más pequeña que puede ser planificada por un sistema operativo. El usuario define el número de *threads*.
Los procesos se inician en el *master thread*
FORK :el *master thread* crea un grupo de threads en paralelo que son ejecutados.
JOIN: cuando los *threads* completan su ejecución, se sincronizan y terminan, dejando solo el *master thread*.



OMP vs MPI


Performance
OpenMP+MPI
MPI
OpenMP
Scalar
Program
Code does not work
Time/Effort

OMP vs MPI


#CPUs
#cores
Dominated by Overhead
MPI
OpenMP
1 core
Scalar
Problem size

Código simple OpenMP

- Constructores OMP son pragma's o directivas de compilación
- OMP fue originalmente creado para paralelizar loops con iteraciones independientes

```
int main(){
double resultado[1000];



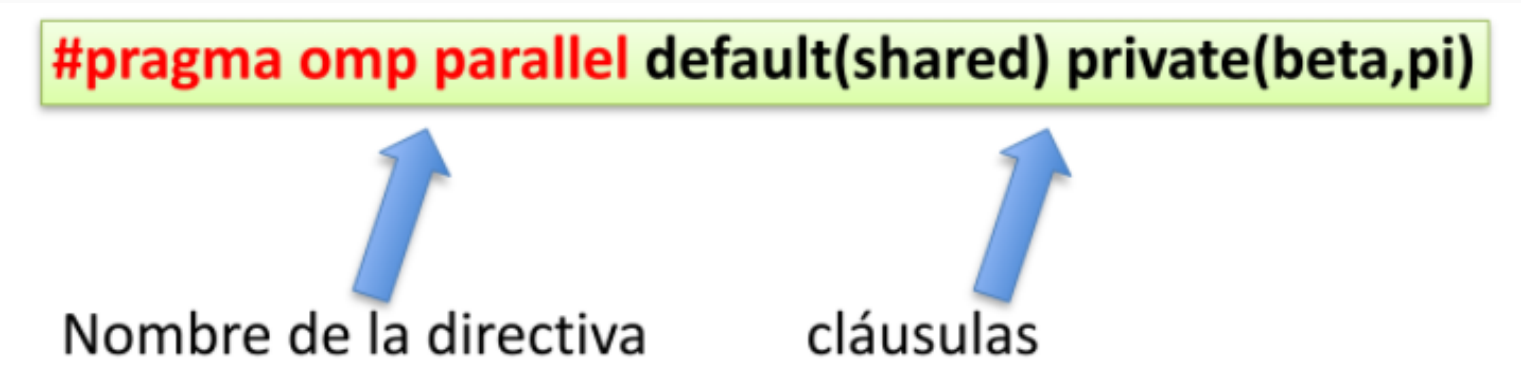
for(int i=0;i<1000;i++)
    calculo_pesado(resultado[i]);


}
```

```
#include<omp.h>
int main(){
double resultado[1000];
// hilo maestro genera
// una region de hilos
#pragma omp parallel for
for(int i=0;i<1000;i++)
    calculo_pesado(resultado[i]);
// los hilos culminan su ejecucion
// y el maestro continua
}
```

Directivas de compilación

Son comentarios sintácticos en el código fuente y seran ignorados por el compilador si el usuario asi lo decide (flag de compilación) Estas inician una región de paralelismo en el código, dividen áreas del código entre threads, distribuyen iteraciones entre threads, secuencializan regiones del código, y sincronizan el trabajo entre threads.



Librerias, Variables de entorno

Librerías son utiles para especificar y alinear el número de *threads*, y sus identificadores, y los *threads* dinámicos, paralelismo entrelazado, y controladores de tiempos de reloj.

```
#include <omp.h>
int omp_get_num_threads(void)
```

Variables de entorno controlan la ejecución en paralelo en tiempo real, como: número de *threads*, cómo están separadas las iteraciones, asignar *threads* a procesos, definiendo paralelismo entrelazado, *threads* dinámicos.

```
csh/tcsh  setenv OMP_NUM_THREADS 8
sh/bash   export OMP_NUM_THREADS=8
```

Compiladores de OpenMP

Compiladores	Lenguaje	Soporte
GCC 4.9.0	C/C++/Fortran	OpenMP 4.0
GCC 9.0	C/C++	OpenMP 5.0
Intel	C/C++/Fortran	OpenMP 5.0
LLVM/Clang 12	C/C++	OpenMP 5.0

Compiler / Platform	Compiler	Flag

Intel
Linux Opteron/Xeon
	`icc`
	`icpc`
	`ifort`
		`-openmp`

PGI
Linux Opteron/Xeon
	`pgcc`
	`pgCC`
	`pgf77`
	`pgf90`
		`-mp`

GNU
Linux Opteron/Xeon
IBM Blue Gene
	`gcc`
	`g++`
	`g77`
	`gfortran`
		`-fopenmp`

IBM
Blue Gene
	`bgxlc_r, bgcc_r`
	`bgxlC_r, bgxlc++_r`
	`bgxlc89_r`
	`bgxlc99_r`
	`bgxlf_r`
	`bgxlf90_r`
	`bgxlf95_r`
	`bgxlf2003_r`
	
	*Be sure to use a thread-safe compiler - its name ends with **_r**
		`-qsmp=omp`

Construcción de regiones en paralelo

Regiones que serán ejecutadas en multiples *threads*, al encontrar la directiva PARALLEL. El master es el *threads* 0, a partir del cual, el código es duplicado y para ser ejecutado por los otros threads. Luego del final de la sección en paralelo, el master *threads* continúa la ejecución

```
#pragma omp parallel [clause ...]
newline
    if(scalar_expression)
    private(list)
    shared(list)
    default (shared | none)
    firstprivate(list)
    reduction(operator: list)
    copyin(list)
    num_threads(integer)

structured_block
```

Construcción de regiones en paralelo

```
#include<omp.h>
int main(){
int nthreads, tid;
// Se genera una region de hilos; cada uno con una variable privada
#pragma omp parallel private(tid)
// se obtiene e imprime cada hilo
tid = omp_get_thread_num();
printf("Hola mundo desde el hilo %d\n" , tid);
// ejecuta solo el maestro:
it(tid==0){
nthreads= omp_get_num_threads();
printf("Numero de hilos: %d\n", nthreads);
} // los hilos culminan su ejecucion y el maestro continua

}
```

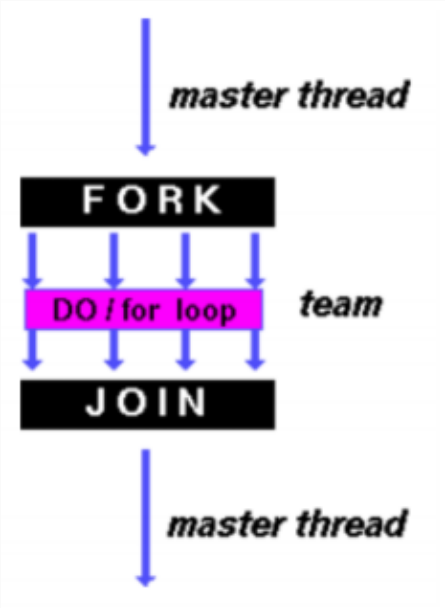
Construcción de regiones en paralelo

Constructor FOR determina que las iteraciones de uno o mas loops asociados se ejecutarán en paralelo en los hilos de la región

```
#pragma omp for [clause ...]
newline
    private(list)
    firstprivate(list)
    lasttprivate(list)
    reduction(operator: list)
    schedule(kind[, chunk_size])
    collapse(n)
    ordered
    nowait
structured_block
```

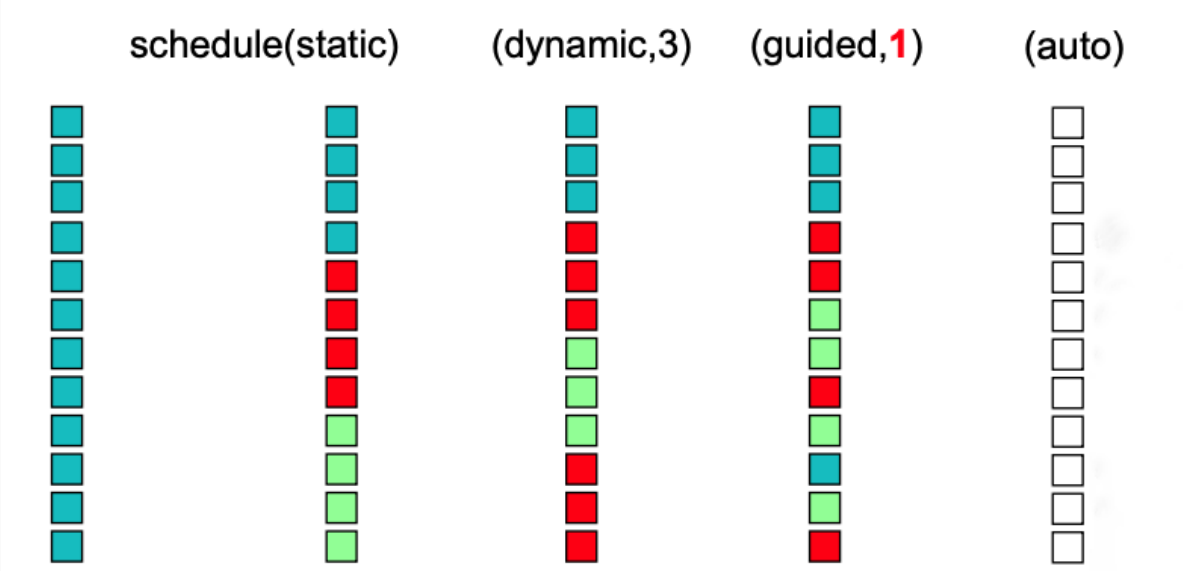
Constructores de trabajo compartido

Dividen la ejecución
de la región comprendida entre los
miembros. Este no crea nuevos *threads*
Constructor
Do/for: Comparte iteraciones en un
loop entre los procesos (*data parallelism*)
SCHEDULE: cláusulas de ordenamiento
de las iteraciones entre procesos.



- **STATIC:** iteraciones son divididas en bloques de tamaño *chunk*, y añadidos a *threads* estáticamente. Si *chunk* no es dado, las iteraciones son divididas igualmente.

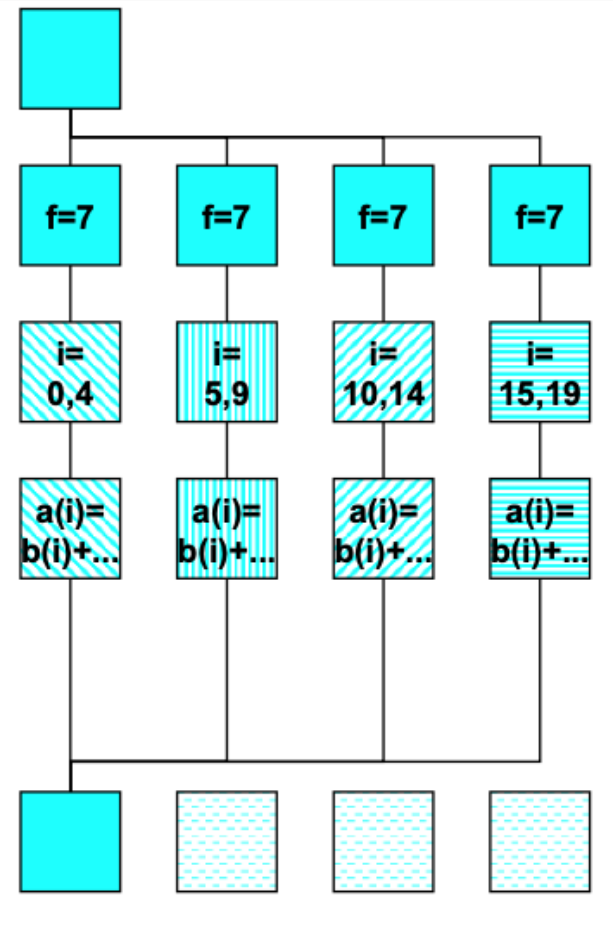
Constructores de trabajo compartido



- **DYNAMIC:** iteraciones son divididas en bloques de tamaño *chunk*, y añadidos a *threads* dinámicamente (apenas culmine su parte)
- **GUIDE:** como DYNAMIC, pero cada bloque disminuye su tamaño (exponencialmente) cada vez que es asignado a un *threads*
- **RUNTIME:** la decisión de ordenamiento se hace en tiempo real por OMP_SCHEDULE (*chunk* no se define aquí)
- **AUTO:** el ordenamiento se delega al compilador

Constructores de trabajo compartido

```
#pragma omp parallel private(f,i)
{
f=7;
#pragma omp for
for(i=0;i<20;i++)
    a[i]=b[i]+f*(i-1);
} // paralel
```



Constructores de trabajo compartido

NO WAIT / nowait:

si especificada, *threads*

no se sincronizan al final del loop en paralelo

```
#include<omp.h>
#define CHUNKSIZE 100
#define N 1000

int main(){
int i, chunk;
float a[N], b[N], c[N];
for(int i=0; i<N, i++)
    a[i]=b[i]=i;
chunk=CHUNKSIZE;

#pragma omp parallel
shared (a,b,c,chunk) private(i)
#pragma omp for
 schedule(dynamic, chunk) nowait
for(int i=0; i<N, i++)
    c[i]=a[i]+b[i];
}
```

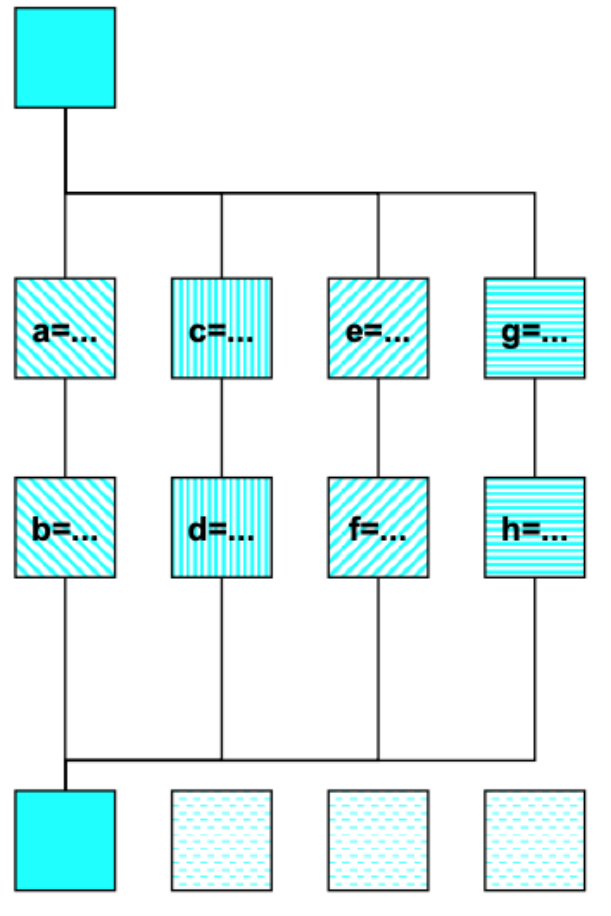
Construcción de regiones en paralelo

SECTIONS: Especifica que secciones del código se dividirán entre los *threads*. Cada SECTION se ejecuta una vez por un *thread*, secciones distintas se ejecutarán por *threads* distintos.
Hay una barrera implícita al final de Sections, a menos que se utilize NOWAIT/nowait
Nota: no usar **goto** en las secciones

```
#pragma omp sections [clause ...]
newline
    private(list)
    firstprivate(list)
    lasttprivate(list)
    reduction(operator: list)
    nowait
{
#pragma omp section newline
structured_block
#pragma omp section newline
structured_block
}
```

Constructores de trabajo compartido

```
#pragma omp parallel
{
#pragma omp sections
{
{ a=...;
  b=...; }
#pragma omp section
{ c=...;
  d=...; }
#pragma omp section
{ e=...;
  f=...; }
#pragma omp section
{ g=...;
  h=...; }
} // sections
} // paralel
```



Construcción de regiones en paralelo

SINGLE: Especifica que la región del código incluída debe ser ejecutada por un solo thread. Se usa, por ejemplo, cuando secciones no son consideradas seguras, como I/O

Threads que no ejecuten SINGLE, esperan hasta el final del bloque, a menos que se use NOWAIT

```
#pragma omp single [clause ...]
newline
    private(list)
    firstprivate(list)
    nowait
structured_block
```

Constructores de trabajo compartido

Pueden combinarse en una sola directiva **PARALLEL**, seguida de directivas de trabajo compartido

```
#include<omp.h>
#define CHUNKSIZE 100
#define N 1000

int main(){
int i, chunk;
float a[N], b[N], c[N];
for(int i=0; i<N, i++){
    a[i]=b[i]=i;
    }
#pragma omp parallel for
shared (a,b,c,chunk) private(i)
schedule(static,chunk)
{
#pragma omp section
for(int i=0; i<N, i++)
    c[i]=a[i]+b[i];
} // paralel
```

Construcción de regiones en paralelo

MASTER: Define una región que debe ser ejecutada solo por el *thread master*. Los otros *threads* evitan esta región.

```
#pragma omp master
newline
structured_block
```

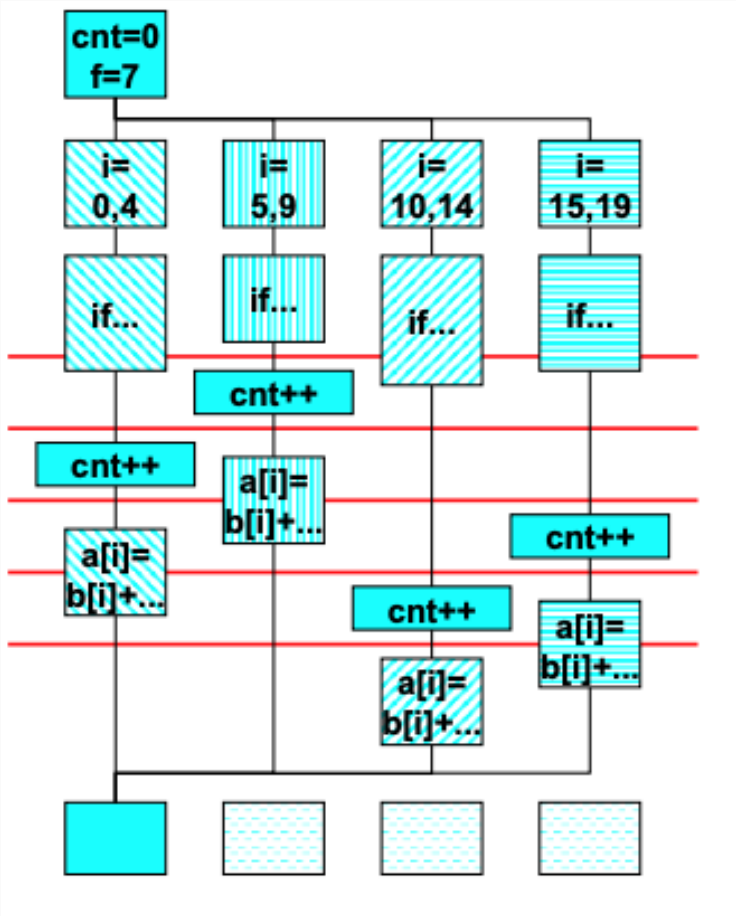
Construcción de regiones en paralelo

CRITICAL: Define una región que debe ser ejecutada por un solo thread a la vez. Si un thread esta ejecutando una region CRITICAL y otro thread alcanza esta región,este último es bloqueado hasta que el primero termine su ejecución.

```
#pragma omp critical
newline
structured_block
```

Constructores de trabajo compartido

Ejemplo de directiva CRITICAL




```
#pragma omp parallel
{
#pragma omp for
{
for(int i=0; i<20, i++){
    if(b[i]==0){
    #pragma omp critical
    cnt++;
    } // if
    a[i]=b[i]+f*(i-1);
} // for
} // paralel
```

Constructores de sincronización

BARRIER: Sincroniza todos los *threads*.Si un *thread* la alcanza, va a esperar a los demas, luego de lo cual, podran seguir ejecutando en paralelo.

```
#pragma omp barrier  newline
```

Constructores de sincronización

ATOMIC: Especifica que la memoria debe ser actualizada en forma atómica, en vez de dejar a otros threads escribir en ella. Actúa como una seccion mini-CRITICAL

Solo se aplica a la linea que le sigue immediatamente

```
#pragma omp atomic newline
  statement_expression
```

Constructores de sincronización

FLUSH: Define un punto de sincronización, en el que el algoritmo debe proveer de una cuantificación consistente de la memoria. Las variables activas de cada *thread* son escritas en la memoria en ese momento.
La lista opcional contiene las variables que van a ser escritas en memoria (*flushed*), para evitar hacerlo con todas las variables. FLUSH se usa particularmente con las siguientes directivas:
Barrier,
Parallel - al inicio y final
Critical – al inicio y final
Ordered – al inicio y final
For – al final
Sections – al final
Single – al final

```
#pragma omp flush (list) newline
```

Constructores de sincronización

ORDERED: Especifica que iteraciones en el loop incluído, seran ejecutados en el mismo orden al de una operacion secuencial *Threads* deberan esperar antes de ejecutar sus bloques de iteraciones (*chunk*) si las iteraciones no han sido aún completadas. Se usa con un DO/for loop Una iteración no debe ejecutar la misma directiva ORDERED mas de una vez, ni ejecutar mas de una directiva ORDERED.

```
#pragma omp for ordered [clauses...]
  (loop region)
#pragma omp ordered  newline
  structured_block
  (endo of loop region)
```

Cláusulas de atributos de alcance de datos (data-sharing)

Dado que la memoria en OPENMP es compartida, las variables tambien lo son, por defecto.
Variables globales, i.e., variables estaticas (*static*) Variables privadas, i.e., indices en loops

Estas directivas, proveen la capacidad de controlar el alcance de datos durante la ejecución de regiones en paralelo (que variables de la region serial del código son trasladadas a la región paralela, y cuales de ellas son visibles para todos los *threads* o son definidas como privadas)

Cláusulas de atributos de alcance de datos (data-sharing)

PRIVATE: Un nuevo objeto del mismo tipo es declarado para cada *thread*.
Toda referencia al objeto original es reemplazado con referencias al nuevo objeto.
Variables privadas no estan originalmente inicializadas para cada *thread*.

SHARED: Declara variables en una lista para ser compartidas entre los *threads*.
Una variable compartida existe solo en una locación de memoria, cuya dirección pueden leer todos los *threads*.
Para acceder a las variables se recomienda usar secciones críticas (CRITICAL).

Cláusulas de atributos de alcance de datos (data-sharing)

DEFAULT: Especifica el alcance por defecto de todas las variables en toda region en paralelo.
Solo puede ser usada una vez en una directiva PARALLEL

FIRSTPRIVATE: Combina PRIVATE con una inicialización automática de las variables en su lista.
El valor de inicialización es el de los objetos originales.
Antes de entrar al constructor de la región en paralelo

Cláusulas de atributos de alcance de datos (data-sharing)

LASTPRIVATE: Combina PRIVATE con una copia de la variable original, de la última iteración o sección.
El valor de la variable original al final de la ejecución es el de la última iteración o sección.

COPYIN: Provee una copia del valor de variables THREADPRIVATE para todos los *threads*.

COPYPRIVATE: Asigna el valor de variables de un *thread* a todas las instancias de las variables privadas en los demas *threads*. Se utiliza con SINGLE.

Cláusulas de atributos de alcance de datos (data-sharing)

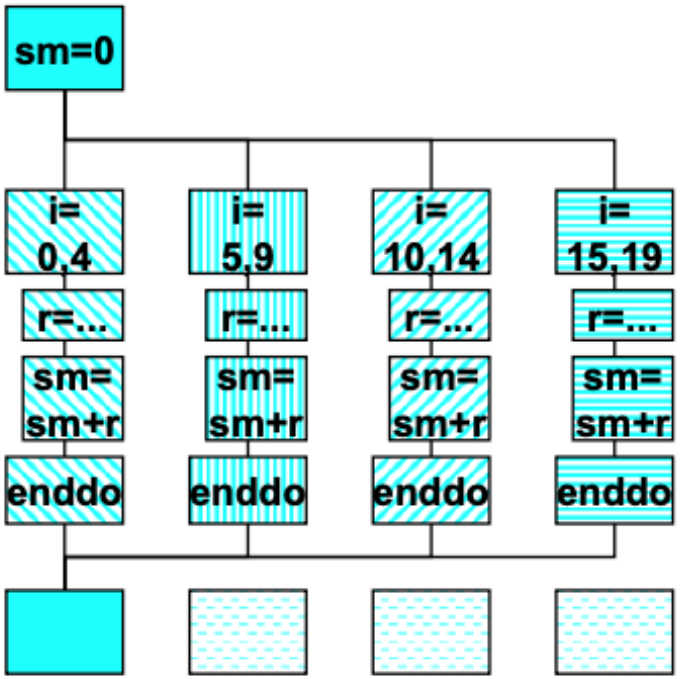
REDUCTION: Realiza una reducción de las variables en la lista. Para ello se copian las variables en cada *thread*. Finalmente, la reducción se aplica a todas las copias privadas de la variable compartida, y el valor final se escribe en la variable global compartida.

C/C++ Reduction Operands

Operator	Initial value
+	0
*	1
-	0
&	~0
|	0
^	0
&&	1
||	0

Constructores de trabajo compartido

```
#pragma omp parallel
{
#pragma omp for reduction(+:sm)
for(int i=0; i<20, i++){
    double r;
    r=work(i);
    sm+=r;
    } //for
} //parallel
```



Constructores de trabajo compartido

Iteraciones en el loop en paralelo se distribuyen en bloques del mismo tamaño a cada *thread* (SCHEDULE STATIC) Al final del loop, todos los *threads* suman su valor a una variable global result, del *master thread*.

Variables en el loop deben ser escalares (no arrays o estructuras), y deben ser declaradas SHARED

```
#pragma omp parallel for
default(shared) private(i)
schedule(static, chunk)
reduction(+:result)

for(int i=0; i<n, i++)
    result+=(a[i]*b[i]);
printf("Final result= \%f\n", result)
```

Cláusulas en directivas OPENMP

Directiva que no aceptan cláusulas:
MASTER
CRITICAL
BARRIER
ATOMIC
FLUSH
ORDERED
THREADPRIVATE

Clause	Directive					
	PARALLEL	DO/for	SECTIONS	SINGLE	PARALLEL DO/for	PARALLEL SECTIONS
IF	●				●	●
PRIVATE	●	●	●	●	●	●
SHARED	●	●			●	●
DEFAULT	●				●	●
FIRSTPRIVATE	●	●	●	●	●	●
LASTPRIVATE		●	●		●	●
REDUCTION	●	●	●		●	●
COPYIN	●				●	●
COPYPRIVATE				●		
SCHEDULE		●			●	
ORDERED		●			●	
NOWAIT		●	●	●		

Bibliografía i

- https://www.openmp.org/
- Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP.* 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.
- Barbara Chapman, Gabriele Jost, and Ruud van der Pas: *Using OpenMP*. MIT Press, 2008, ISBN-13: 978-0-262-53302-7.
- Georg Hager, Gerhard Wellein: *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, 2010, 256p, ISBN13: 978-1-439-81192-4
- R.Chandra, L. Dagum, D. Kohr, D. Maydan, J. McDonald, and R. Menon: *Parallel programming in OpenMP*. Academic Press, San Diego, USA, 2000, ISBN 1-55860-671-8

UTEC
UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

